Guión de trabajo tutorizado

Sistemas de Recuperación de Información

Gestión y visualización de datos masivos

Profesor D. Alejandro Gómez Morón
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA-Recuperación de Información (Solr)-Actividad.pdf» (Campus Virtual UPO).

Objetivos

- Ser capaz de realizar una instalación básica de Solr.
- Crear y consultar datos a nivel básico usando Solr.
- Conocer los beneficios de utilizar un Sistema de Recuperación de Información.

Actividades

A1. (75 minutos - 2 puntos) Realizaremos una instalación básica de Apache Solr. Para ello, leeremos detenidamente y seguiremos el tutorial oficial de Solr [1].

Como prueba de realización de esta actividad deberá aportar dos capturas de pantalla:

1. Salida del comando bin/solr status
2. Salida que obtiene en el navegador en http://localhost:8983/solr/

Como observará al hacer la lectura, el entorno de despliegue de este tipo de motores de bases de datos es un entorno Linux. Aunque existe posibilidad de desplegarlo en sistemas Windows para pruebas, no son tan eficientes como en sistemas Linux, así que, en caso de que el sistema operativo que tenga sea Windows, deberá crear una máquina virtual para hacer este ejercicio (y los siguientes). Para ello puede seguir cualquier tutorial al efecto, como el de [2]. También puede utilizar contenedores y levantar un container de Solr (ambas opciones son válidas).

Tenga en cuenta que también existen soluciones de Apache Solr dockerizadas y son posibles utilizarlas. En caso que elija esta opción, tiene que entrar en el contenedor e indicar la salida que le sale en el comando indicado anteriormente así como el comando de docker utilizado para levantar el contenedor.

A2. (165 minutos - 4 puntos) A continuación vamos a seguir el tutorial de Apache Solr, concretamente el ejercicio 0 [6]. Este ejercicio nos permitirá crear nuestro esquema para el ejercicio siguiente. Además haremos alguna prueba de indexación tal y como dice el tutorial.

Nota: Durante el tutorial verá ejecuciones de un comando llamado curl. Si no tuviese el programa disponible en su Linux, puede instalarlo ejecutando sudo apt-get install curl.

Antes que llegue a la sección del ejercicio 1 del tutorial (titulada Index Techproducts Data) [7], haga una captura de pantalla de la interfaz de administración de Solr con la salida de http://localhost:8983/solr/#/~cloud, pues será un entregable de esta actividad.

Una vez hecha la captura, prosiga con el indexado de datos [7], donde realmente lo que hará Solr es guardar esos datos y mapearlos al schema que se creó con el core.

Con los datos ya almacenados, busque el libro titulado “The Black Company”. Como parte del entregable de esta actividad conteste a estas preguntas: ¿Qué resultado obtiene? ¿Por qué?

Una vez hecha esta búsqueda de término simple, realice una búsqueda de la cadena “System Memory”, sin centrarse en ningún campo. Como parte del entregable de esta actividad conteste a estas preguntas: ¿Qué resultado obtiene? ¿Por qué? ¿Qué diferencia esta búsqueda de la anterior?

A la búsqueda anterior, mejórela con una búsqueda facetada por el campo del género. Como parte del entregable de esta actividad conteste a estas preguntas: ¿Qué resultado obtiene? ¿Por qué? ¿Qué diferencia esta búsqueda de la anterior? ¿Cuándo usaría esta búsqueda y cuándo la otra?

A3. (180 minutos - 4 puntos) En esta actividad practicaremos y analizaremos cómo es el proceso de análisis de las consultas que realizó en las actividades anteriores.

Como prueba de la realización de la actividad se creará una sección en la entrega con un contenido similar al realizado para la actividad 1 y 2, siendo necesario que, por cada consulta que realice, utilice el analizador y aporte, en un documento, explicación de cómo una consulta con unos valores de entrada es procesada internamente (centrándose en la los filtros de tokenización y de stop como vio en la transparencia 31 y la documentación que puede encontrar en [9])

Realice la búsqueda de las cadenas: System, Company Cook, Company with Cook.

Conteste, además, a las siguientes preguntas:

1. ¿Qué ocurriría si, en el esquema, el campo name no fuese indexed? ¿Cambiaría el resultado de la búsqueda? ¿Por qué?

2. ¿Qué ocurriría si, en el esquema, el campo fuese indexed pero en Español? ¿Cambiaría la búsqueda? ¿Por qué?

Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- La entrega se ha de llevar a cabo durante el día 20 de abril de 2026.

Bibliografía

1. Apache Solr ™ Instalación https://solr.apache.org/guide/solr/latest/deployment-guide/installing-solr.html

2. Ask Ubuntu: How to install Ubuntu on VirtualBox http://askubuntu.com/questions/142549/how-to-install-ubuntu-on-virtualbox https://brb.nci.nih.gov/seqtools/installUbuntu.html

3. Apache Solr – System Requirements https://solr.apache.org/guide/solr/latest/deployment-guide/system-requirements.html

4. Apache Solr – Highlighting https://solr.apache.org/guide/solr/latest/query-guide/highlighting.html

5. Apache Solr – Facet Parameters https://solr.apache.org/guide/solr/latest/query-guide/faceting.html

6. Apache Solr – Tutorial (Ejercicio 0) https://solr.apache.org/guide/solr/latest/getting-started/tutorial-five-minutes.html

7. Apache Solr – Tutorial (Ejercicio 1, indexación de datos) https://solr.apache.org/guide/solr/latest/getting-started/tutorial-techproducts.html

8. Apache Solr - Sobre tokenizadores https://solr.apache.org/guide/solr/latest/indexing-guide/tokenizers.html#about-tokenizers

RESOLUCIÓN

Entorno utilizado

Máquina Linux (Ubuntu 24.04) con OpenJDK 17 y **Apache Solr 9.9.0**, la distribución binaria completa `solr-9.9.0.tgz` descargada del archivo oficial de Apache y descomprimida en `/opt/solr`. Se trabaja en **modo SolrCloud** (`-c`), que arranca un ZooKeeper embebido en el puerto 9983, como en el tutorial oficial.

A1. Instalación básica de Apache Solr

Tras descomprimir el paquete, Solr se arranca con `bin/solr start -c`. El parámetro `--force` sólo es necesario porque la máquina de pruebas trabaja como `root`; con un usuario normal no hace falta. Solr espera a que el nodo esté disponible y confirma el puerto (8983) y el PID.

```
aramber@localhost:~$ bin/solr start -c --force
Solr will start in SolrCloud mode by default in version 10, and you will no longer need to pass in -c or --cloud flag.

Java 17 detected. Enabled workaround for SOLR-16463
Waiting up to 180 seconds to see Solr running on port 8983 [|]  [/]  [-]  [\]  [|]
Started Solr server on port 8983 (pid=12956). Happy searching!

    
```

Figura 1. Arranque de Solr en modo SolrCloud.

Captura 1 – `bin/solr status`: el proceso está activo en el puerto 8983 y la salida JSON muestra la ruta `solr_home`, la versión 9.9.0, el tiempo en marcha, la memoria (512 MB de heap) y el bloque `cloud`: ZooKeeper en `127.0.0.1:9983`, 1 nodo vivo y todavía 0 colecciones.

```
aramber@localhost:~$ bin/solr status

Solr process 12956 running on port 8983
{
  "solr_home":"/opt/bd/solr-9.9.0/server/solr",
  "version":"9.9.0 386ed096a1946c488cfe576a19a147bdb1153508 - houston - 2025-07-18 14:15:23",
  "startTime":"Sat Sep 19 12:00:48 CEST 2026",
  "uptime":"0 days, 0 hours, 0 minutes, 9 seconds",
  "memory":"144.7 MB (%28.3) of 512 MB",
  "cloud":{
    "ZooKeeper":"127.0.0.1:9983",
    "liveNodes":"1",
    "collections":"0"}}

WARNING: URLs provided to this tool needn't include Solr's context-root (e.g. "/solr"). Such URLs are deprecated and s
upport for them will be removed in a future release. Correcting from [http://localhost:8983/solr/] to [http://localhos
t:8983/].
```

Figura 2. Salida de `bin/solr status`.

Captura 2 – http://localhost:8983/solr/: el *Dashboard* de la consola de administración con las versiones de Solr y Lucene (9.12.2), los argumentos de la JVM y el uso de memoria. La barra lateral incluye la sección **Cloud**, propia del modo SolrCloud.

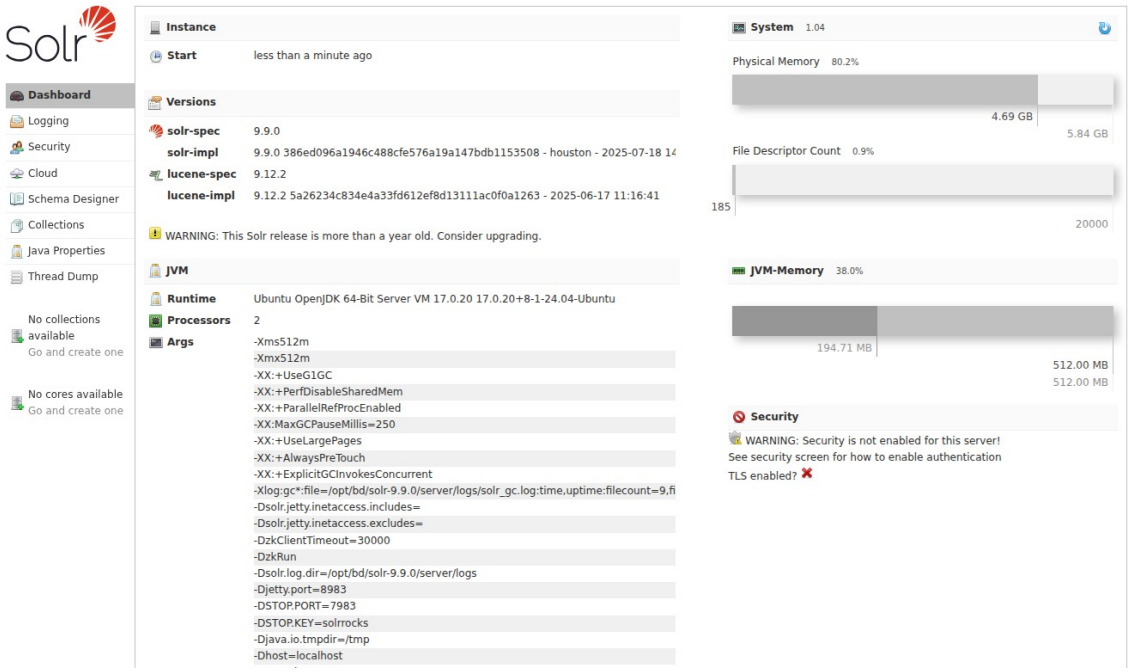


Figura 3. Consola de administración de Solr (Dashboard).

A2. Tutorial: creación de la colección, indexación y búsquedas

Ejercicio 0 – creación del esquema (colección techproducts)

Se crea la colección `techproducts` con el *configset* `sample_techproducts_configs`, que incluye el esquema de campos del tutorial. Se reparte en **2 shards con 2 réplicas** cada uno, igual que en el tutorial.

```
aramber@localhost:~$ bin/solr create -c techproducts --conf-dir sample_techproducts_configs --shards 2 --replication-factor 2
Created collection 'techproducts' with 2 shard(s), 2 replica(s) with config-set 'techproducts'
Neither --zk-host or --solr-url parameters provided so assuming solr url is http://localhost:8983.
Neither --zk-host or --solr-url parameters provided so assuming solr url is http://localhost:8983.
WARN  - 2026-09-19 12:01:11.796; org.apache.solr.common.cloud.SolrZkClient; Using default ZkCredentialsInjector. ZkCre
dentialsInjector is not secure, it creates an empty list of credentials which leads to 'OPEN_ACL_UNSAFE' ACLs to Zooke
eper nodes
WARN  - 2026-09-19 12:01:11.841; org.apache.solr.common.cloud.SolrZkClient; Using default ZkACLProvider. DefaultZkACLP
rovider is not secure, it creates 'OPEN_ACL_UNSAFE' ACLs to Zookeeper nodes
WARNING: URLs provided to this tool needn't include Solr's context-root (e.g. "/solr"). Such URLs are deprecated and s
upport for them will be removed in a future release. Correcting from [http://localhost:8983/solr] to [http://localhost
:8983/].
```

Figura 4. `bin/solr create` de la colección techproducts (2 shards × 2 réplicas). Los avisos WARN sobre ACL de ZooKeeper son informativos en un entorno local.

Entregable – `http://localhost:8983/solr/#/~cloud`: la vista *Nodes* muestra el nodo `8983_solr` alojando las réplicas de techproducts. La vista *Graph* muestra la topología: la colección se divide en `shard1` y `shard2`, cada uno con dos réplicas; la marcada con punto negro es la líder.

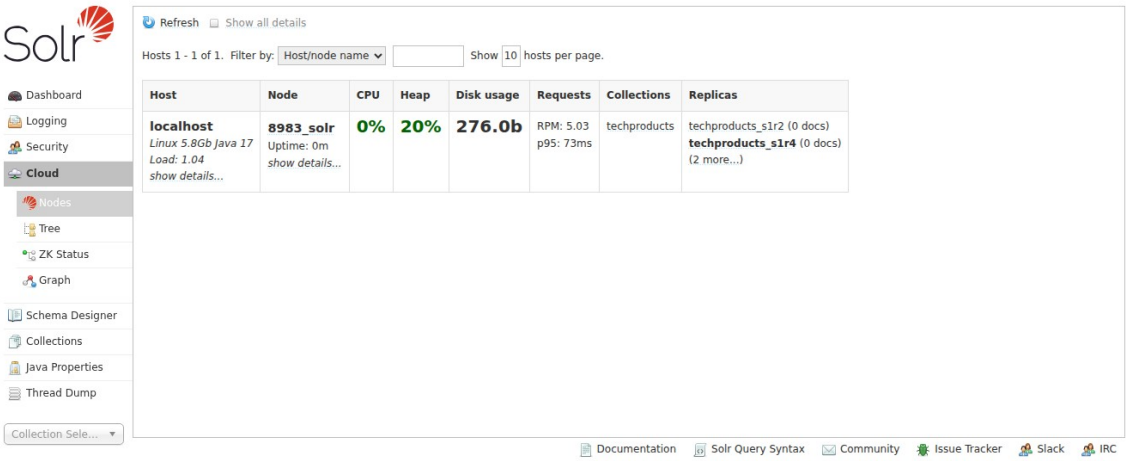


Figura 5. Cloud › Nodes (captura realizada antes del indexado de datos).



Figura 6. Cloud › Graph: techproducts con 2 shards y 2 réplicas por shard.

Ejercicio 1 – Index Techproducts Data

Se indexan los documentos de ejemplo con la herramienta `bin/solr post`. Solr lee cada fichero (XML, JSON, CSV), convierte sus campos al esquema de la colección (p. ej. `name` → `text_general`, `genre_s` → `string`, `price` → `pfloat`), construye el índice invertido y, al hacer *commit*, deja los documentos disponibles para búsqueda. Se omiten los PDF/HTML de la carpeta de ejemplos porque requieren el módulo *extraction* (Tika), que no se usa en esta actividad. Resultado: **45 documentos**.

```
aramber@localhost:~$ bin/solr post -c techproducts example/exampledocs/*.xml example/exampledocs/*.json example/exampledocs/*.csv 2>&1 |
grep -v WARN
Posting files to [base] url http://localhost:8983/solr/techproducts/update...
Entering auto mode. File endings considered are xml,json,jsonl,csv,pdf,doc,docx,ppt,pptx,xls,xlsx,odt,odp,ods,ott,otp,
ots,rtf,htm,html,txt,log
POSTing file gb18030-example.xml (application/xml) to [base]
POSTing file hd.xml (application/xml) to [base]
POSTing file ipod_other.xml (application/xml) to [base]
POSTing file ipod_video.xml (application/xml) to [base]
POSTing file manufacturers.xml (application/xml) to [base]
POSTing file mem.xml (application/xml) to [base]
POSTing file money.xml (application/xml) to [base]
POSTing file monitor.xml (application/xml) to [base]
POSTing file monitor2.xml (application/xml) to [base]
POSTing file mp500.xml (application/xml) to [base]
POSTing file sd500.xml (application/xml) to [base]
POSTing file solr.xml (application/xml) to [base]
POSTing file vidcard.xml (application/xml) to [base]
POSTing file books.json (application/json) to [base]/json/docs
POSTing file books.csv (text/csv) to [base]
1 files indexed.
COMMITting Solr index changes to http://localhost:8983/solr/techproducts/update...
Time spent: 0:00:01.062
aramber@localhost:~$ curl -s 'http://localhost:8983/solr/techproducts/select?q=*:*&rows=0' | grep numFound
    "numFound":45,
    "numFoundExact":true,
```

Figura 7. Indexación de los documentos de ejemplo y comprobación del número de documentos (`numFound: 45`).

Búsqueda 1: «The Black Company»

```
aramber@localhost:~$ curl -s 'http://localhost:8983/solr/techproducts/select?q=The%20Black%20Company&fl=id,name,genre_s,score&rows=10'
{
  "responseHeader":{
    "zkConnected":true,
    "status":0,
    "QTime":20,
    "params":{
      "q":"The Black Company",
      "fl":"id,name,genre_s,score",
      "rows":"10"
    }
  },
  "response":{
    "numFound":9,
    "start":0,
    "maxScore":3.4597433,
    "numFoundExact":true,
    "docs":[{
      "id":"0812521390",
      "name":"The Black Company",
      "genre_s":"fantasy",
      "score":3.4597433
    },{
      "id":"080508049X",
      "name":"The Black Cauldron",
      "genre_s":"fantasy",
      "score":1.886039
    },{
      "id":"978-1423103349",
      "score":1.0900079
    },{
      "id":"978-1857995879",
      "score":0.8705727
    },{
      "id":"978-0641723445",
      "score":0.86629874
    },{
      "id":"0805080481",
      "name":"The Book of Three",
      "genre_s":"fantasy",
      "score":0.83955353
    },{
      "id":"SOLR1000",
      "name":"Solr, the Enterprise Search Server",
      "score":0.71804243
    },{
      "id":"0579B002",
      "name":"Canon PIXMA MP500 All-In-One Photo Printer",
      "score":0.65560734
    },{
      "id":"MA147LL/A",
      "name":"Apple 60 GB iPod with Video Playback Black",
      "score":0.3252257
    }]
  }
}
```

Figura 8. `q=The Black Company` sin especificar campo.

¿Qué resultado obtiene? 9 documentos. El primero, con una puntuación muy superior (3,46), es el libro *The Black Company* (id `0812521390`, género *fantasy*). Detrás aparecen *The Black Cauldron*, varios libros de `books.json`, el libro sobre Solr, una impresora Canon y un iPod negro.

¿Por qué? Sin campo explícito, la consulta se ejecuta contra el campo por defecto (`df=text`), un campo *catch-all* que recibe por `copyField` el nombre, el autor, la categoría,

las características, etc. El analizador divide la consulta en tres términos (`the`, `black`, `company`) y los une con el operador por defecto **OR**. Por eso basta con que un documento contenga *alguno* de ellos: el iPod entra por `black` y otros por `the`. La ordenación por relevancia (BM25) premia a los documentos que contienen más términos y términos más raros. *The Black Company* los tiene todos y `company` es poco frecuente en la colección, así que queda muy por delante. Si se quiere sólo ese libro, hay que buscar la **frase** en el campo del título:



Figura 9. Búsqueda de frase en el campo `name`: un único resultado exacto (autor Glen Cook).

Búsqueda 2: «System Memory»

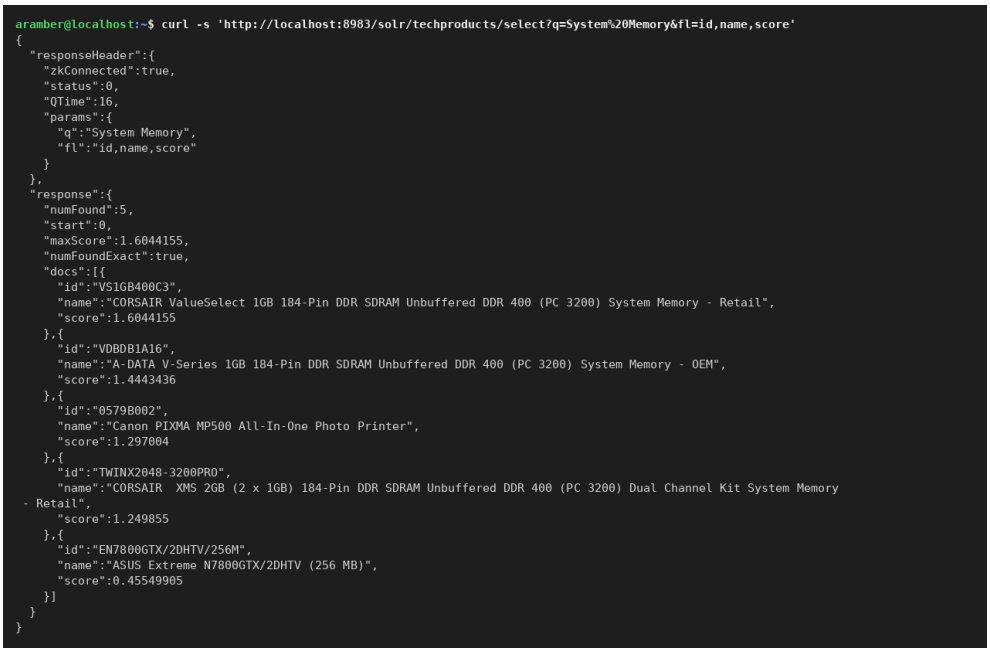


Figura 10. `q=System Memory` sin centrarse en ningún campo.

Resultado: 5 documentos. Los tres primeros son módulos de memoria (Corsair ValueSelect, A-DATA, Corsair XMS), cuyo nombre contiene literalmente «System Memory»; también aparecen una impresora multifunción Canon y una tarjeta gráfica ASUS. **¿Por qué?** Igual que antes, se buscan `system` OR `memory` en el campo `text`. La impresora y la gráfica aparecen porque uno de sus campos (características o categoría) contiene el término `memory`, pero no `system`, y quedan con menor puntuación. **Diferencia con la búsqueda anterior:** allí el

objetivo era un título concreto y un único documento concentraba toda la relevancia. Aquí la consulta describe una *categoría* de producto, hay varios documentos igual de pertinentes y los términos aparecen en campos descriptivos, no sólo en el título.

Búsqueda 3: «System Memory» con búsqueda facetada por género

```
aramber@localhost:~$ curl -s 'http://localhost:8983/solr/techproducts/select?q=System%20Memory&fl=id,name&rows=0&facet=true&facet.field=g
enre_s&facet.mincount=0'
{
  "responseHeader":{
    "zkConnected":true,
    "status":0,
    "QTime":30,
    "params":{
      "q":"System Memory",
      "facet.field":"genre_s",
      "fl":"id,name",
      "facet.mincount":"0",
      "rows":"0",
      "facet":"true"
    }
  },
  "response":{
    "numFound":5,
    "start":0,
    "maxScore":1.6044155,
    "numFoundExact":true,
    "docs":[ ]
  },
  "facet_counts":{
    "facet_queries":{ },
    "facet_fields":{
      "genre_s":["fantasy",0,"scifi",0]
    },
    "facet_ranges":{ },
    "facet_intervals":{ },
    "facet_heatmaps":{ }
  }
}
```

Figura 11. `facet.field=genre_s` sobre la búsqueda anterior (`facet.mincount=0` para ver todos los valores).

Resultado: la búsqueda devuelve los mismos 5 documentos, pero la faceta `genre_s` sale **vacía: fantasy 0, scifi 0**. **¿Por qué?** El campo género sólo existe en los libros (`books.csv`/`books.json`); ninguno de los productos de memoria lo tiene, así que no hay valores que contar. La faceta cuenta cuántos documentos del resultado tienen cada valor de un campo; no filtra ni cambia el *ranking*. Si se facetara por un campo que sí tienen estos productos, como `cat`, sí sería útil:

```
aramber@localhost:~$ curl -s 'http://localhost:8983/solr/techproducts/select?q=System%20Memory&rows=0&facet=true&facet.field=genre_s&face
t.field=cat&facet.mincount=1&omitHeader=true'
{
  "response":{
    "numFound":5,
    "start":0,
    "maxScore":1.6044155,
    "numFoundExact":true,
    "docs":[ ]
  },
  "facet_counts":{
    "facet_queries":{ },
    "facet_fields":{
      "genre_s":[ ],
      "cat":["electronics",5,"memory",3,"copier",1,"graphics card",1,"multifunction printer",1,"printer",1,"scanner",1
]
    },
    "facet_ranges":{ },
    "facet_intervals":{ },
    "facet_heatmaps":{ }
  }
}
```

Figura 12. Misma consulta facetando por `genre_s` y `cat` (sólo valores con al menos 1 documento).

La faceta `cat` revela que los 5 resultados son *electronics*, 3 de ellos *memory*, y el resto impresora, escáner, copiadora o tarjeta gráfica. Con eso el usuario puede refinar con un filtro (`fq=cat:memory`) y quedarse con los 3 módulos. **Diferencia con la anterior:** la búsqueda simple devuelve una lista ordenada; la facetada añade un resumen agregado del conjunto de resultados que sirve para navegar y filtrar. **¿Cuándo usar cada una?** La búsqueda simple, cuando el usuario sabe lo que busca (un título, una referencia). La facetada, en búsquedas exploratorias con muchos resultados heterogéneos (catálogos de tienda online, bibliotecas,

buscadores de empleo), sobre campos categóricos que realmente tengan los documentos: el género para libros, la categoría o el fabricante para electrónica.

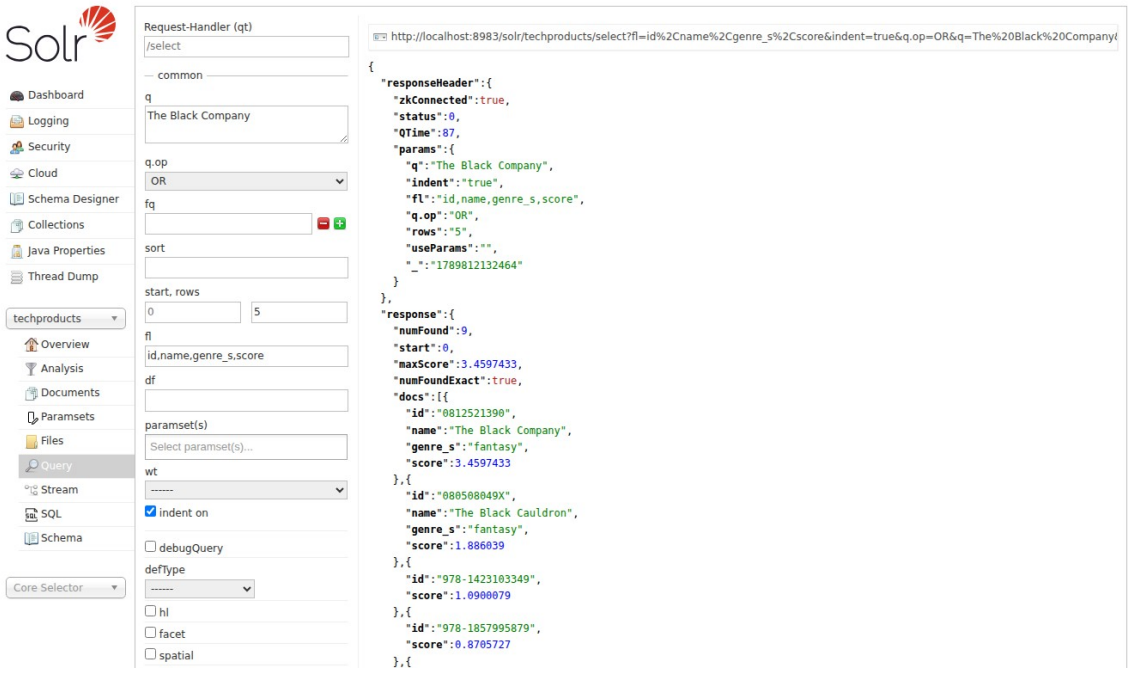


Figura 13. La búsqueda de «The Black Company» ejecutada desde la pantalla *Query* de la consola de administración.

A3. Análisis de las consultas

Para cada cadena se ha usado la pantalla **Analysis** sobre el campo `name`, de tipo `text_general`, y se ha ejecutado la búsqueda con `debug=query` para ver la consulta que genera Solr (`parsedquery`). La cadena usada en la parte de indexación es «The Black Company by Glen Cook - System Memory», elegida para que contenga todos los términos buscados. La cadena de indexación de `text_general` es: **StandardTokenizer (ST) → StopFilter (SF) → LowerCaseFilter (LCF)**. La de consulta añade **SynonymGraphFilter (SGF)** antes de pasar a minúsculas.

«System»

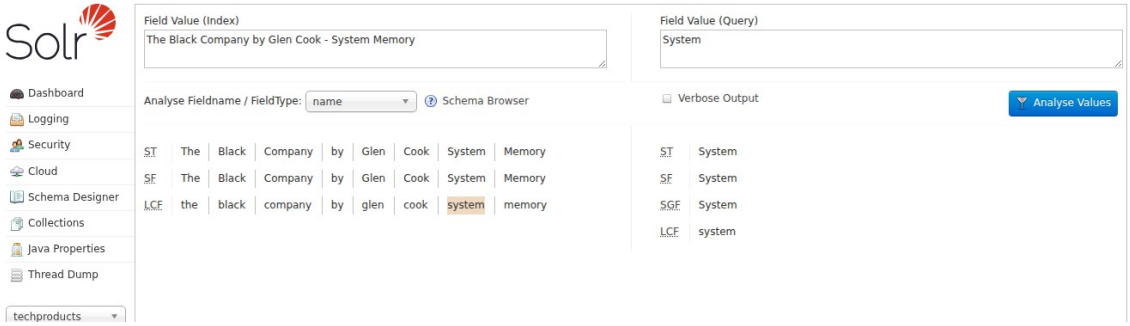


Figura 14. Análisis de «System» en el campo `name`.

```
aramber@localhost:~$ curl -s 'http://localhost:8983/solr/techproducts/select?q=System&fl=id,name,author,score&debug=query&omitHeader=true
' | jq '{numFound: .response.numFound, docs: .response.docs, parsedquery: .debug.parsedquery}'
{
  "numFound": 3,
  "docs": [
    {
      "id": "VS1GB400C3",
      "name": "CORSAIR ValueSelect 1GB 184-Pin DDR SDRAM Unbuffered DDR 400 (PC 3200) System Memory - Retail",
      "score": 0.74938774
    },
    {
      "id": "VDBDB1A16",
      "name": "A-DATA V-Series 1GB 184-Pin DDR SDRAM Unbuffered DDR 400 (PC 3200) System Memory - OEM",
      "score": 0.66068614
    },
    {
      "id": "TWINX2048-3200PRO",
      "name": "CORSAIR  XMS 2GB (2 x 1GB) 184-Pin DDR SDRAM Unbuffered DDR 400 (PC 3200) Dual Channel Kit System Memor
y - Retail",
      "score": 0.55706966
    }
  ],
  "parsedquery": "text:system"
}
```

Figura 15. Búsqueda `q=System`: 3 resultados y consulta generada `text:system`.

Tokenización: el StandardTokenizer produce un único token `System`. El StopFilter no lo elimina, porque no está en la lista de palabras vacías. Tampoco hay sinónimos para él. El LowerCaseFilter lo convierte en `system`, que coincide (resaltado) con el token `system` indexado. La consulta resultante es `text:system` y devuelve los 3 módulos de memoria cuyo nombre contiene «System».

«Company Cook»

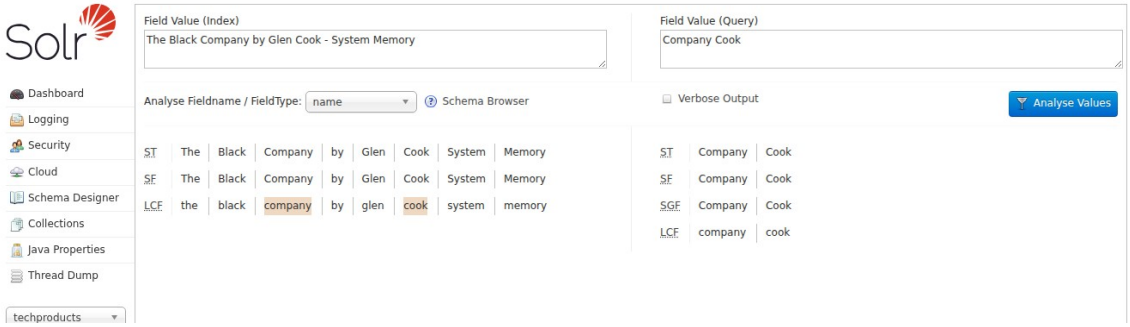


Figura 16. Análisis de «Company Cook».

```
aramber@localhost:~$ curl -s 'http://localhost:8983/solr/techproducts/select?q=Company%20Cook&fl=id,name,author,score&debug=query&omitHea
der=true' | jq '{numFound: .response.numFound, docs: .response.docs, parsedquery: .debug.parsedquery}'
{
  "numFound": 1,
  "docs": [
    {
      "id": "0812521390",
      "name": "The Black Company",
      "author": "Glen Cook",
      "score": 3.1474085
    }
  ],
  "parsedquery": "text:company text:cook"
}
```

Figura 17. Búsqueda `q=Company Cook`: 1 resultado, consulta `text:company text:cook`.

Dos tokens, `Company` y `Cook`, que tras el paso a minúsculas coinciden con los tokens indexados `company` y `cook`. La consulta es la disyunción `text:company text:cook`. Devuelve sólo *The Black Company*, porque es el único documento con `company` (título) y con `cook` (autor Glen Cook, copiado al campo `text`).

«Company with Cook»

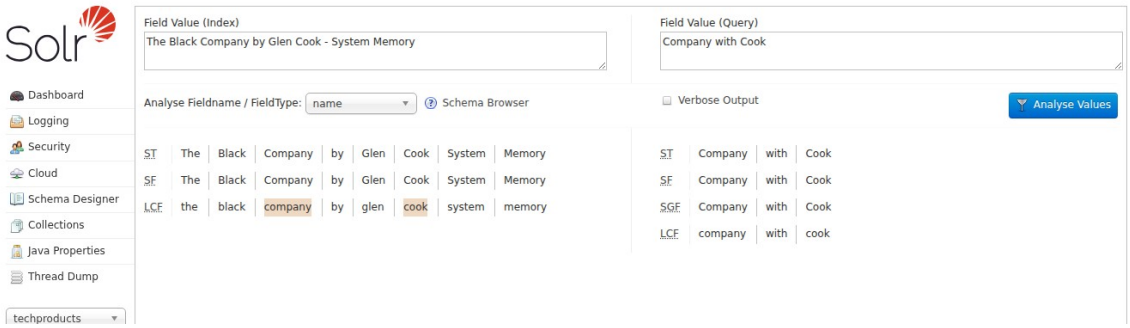


Figura 18. Análisis de «Company with Cook»: el token `with` atraviesa el StopFilter.

```
aramber@localhost:~$ curl -s 'http://localhost:8983/solr/techproducts/select?q=Company%20with%20Cook&fl=id,name,author,score&debug=query&
omitHeader=true' | jq '{numFound: .response.numFound, docs: .response.docs, parsedquery: .debug.parsedquery}'
{
  "numFound": 4,
  "docs": [
    {
      "id": "0812521390",
      "name": "The Black Company",
      "author": "Glen Cook",
      "score": 3.1474085
    },
    {
      "id": "SOLR1000",
      "name": "Solr, the Enterprise Search Server",
      "score": 1.0776832
    },
    {
      "id": "GB18030TEST",
      "name": "Test with some GB18030 encoded characters",
      "score": 0.70585895
    },
    {
      "id": "MA147LL/A",
      "name": "Apple 60 GB iPod with Video Playback Black",
      "score": 0.65428996
    }
  ],
  "parsedquery": "text:company text:with text:cook"
}
```

Figura 19. Búsqueda `q=Company with Cook`: 4 resultados, consulta `text:company text:with text:cook`.

Aquí está la diferencia importante: **el token `with` no se elimina en el StopFilter**. El fichero `stopwords.txt` del configset de ejemplo está vacío (sólo tiene comentarios), así que el filtro no tiene palabras que quitar. La consulta pasa a ser `text:company OR text:with OR text:cook` y aparecen 3 documentos más que sólo contienen «with» (el libro de Solr, un documento de prueba GB18030 y el iPod «with Video Playback»). *The Black Company* sigue primero porque es el único que casa con los tres términos. Con una lista de palabras vacías en inglés (p. ej. `lang/stopwords_en.txt`), `with` se descartaría y el resultado sería el mismo que en «Company Cook».

Preguntas

1. ¿Qué ocurriría si, en el esquema, el campo `name` no fuese `indexed`? No se construiría índice invertido para ese campo: su valor se guardaría (si `stored=true`) para devolverlo en los resultados, pero no sería buscable. Una consulta dirigida al campo, como `name:"The Black Company"`, no devolvería nada (o daría error si además no tiene *docValues*). **Las búsquedas de esta actividad no cambiarían**, porque se lanzan sin campo contra `text`, que recibe una copia de `name` mediante `copyField` y sí está indexado. El `copyField` copia el valor original antes de analizarlo, así que no depende de que el campo origen esté indexado. Sí cambiaría cualquier búsqueda, ordenación o faceta que usara explícitamente `name`.

2. ¿Qué ocurriría si el campo fuese `indexed` pero en español? Se usaría un tipo como `text_es`, con otra cadena de análisis: minúsculas, palabras vacías en español (`de`, `la`, `con`…) y un *stemmer* ligero en español. En la figura se ve que para estas cadenas en inglés el resultado apenas cambia (`company`, `with`, `cook` quedan igual, porque el stemmer español no los reconoce). Sin embargo:

- Las **palabras vacías en inglés no se eliminarían**, porque la lista es de español: `with`, `the` o `by` seguirían generando coincidencias como las vistas arriba.
- Las **variantes morfológicas en inglés no se unificarían**: un *stemmer* inglés reduce «companies» a «compani», igual que «company», pero el español no. Una búsqueda en plural no encontraría el singular.
- Aplicado a textos en inglés, el stemmer español podría recortar sufijos que coinciden por casualidad (terminaciones en -s, -es, -a, -o) y producir coincidencias erróneas. Lo importante es que **índice y consulta usen el mismo analizador y el adecuado al idioma del contenido**. Si no, los términos generados no coinciden y la búsqueda pierde exhaustividad o precisión.

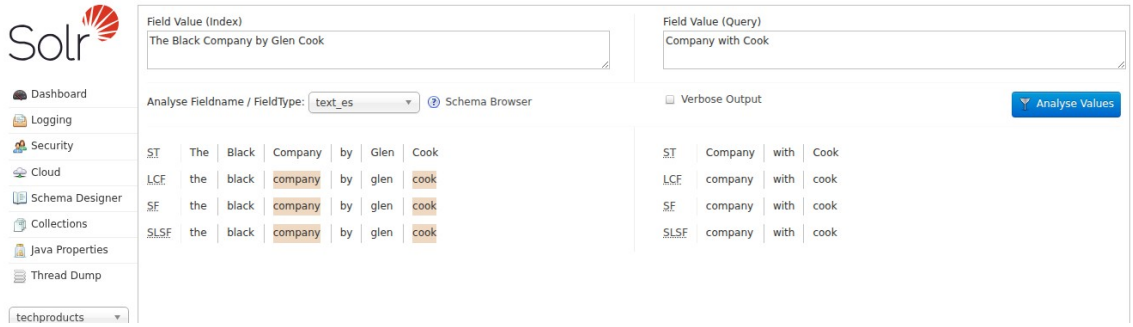


Figura 20. La misma consulta analizada con el tipo `text_es`: minúsculas, StopFilter en español y SpanishLightStemFilter (SLSF).

Referencias

[1] Apache Solr Reference Guide 9.9 – Installing Solr / Tutorials (Exercise 0 y Techproducts). https://solr.apache.org/guide/solr/latest/

[2] Apache Solr – Faceting. https://solr.apache.org/guide/solr/latest/query-guide/faceting.html

[3] Apache Solr – Analyzers, Tokenizers and Filters. https://solr.apache.org/guide/solr/latest/indexing-guide/tokenizers.html